

الدليل الشامل في تحليل البيانات باستخدام Python و Pandas

هذا الدليل يلخص خطوات دورة العمل الكاملة لتحليل البيانات (Data Analysis Pipeline)، بناءً على المحاضرة، بدءاً من استيراد البيانات الخام وحتى بناء اللوحات التفاعلية وتصديرها. الدليل مقسم إلى خطوات منطقية مع الشرح والأكواد البرمجية المناسبة.

1. إعداد البيئة وقراءة البيانات (Setup & Data Loading)

الخطوة الأولى في أي مشروع تحليل بيانات هي استيراد المكتبات الأساسية وقراءة البيانات للتعرف على هيكلها.

استيراد المكتبات الأساسية

نحتاج إلى مكتبات قوية للتعامل مع البيانات والرياضيات والرسوم البيانية: **pandas** : للتعامل مع الجداول وتحليلها. **numpy** : للعمليات الحسابية المتقدمة. **matplotlib** و **seaborn** : لإنشاء رسوم بيانية ثابتة. **scipy** : للتحليلات الإحصائية. **plotly** : للرسوم البيانية التفاعلية.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import plotly.express as px
import plotly.graph_objects as go
```

قراءة الملف واستكشاف البيانات المبدئي

نقوم بقراءة البيانات من ملف **CSV** واستكشافها للتحقق من الأعمدة والأنواع.

```
# قراءة البيانات
file_df = 'OmairData.csv'
df = pd.read_csv(file_df)
```

```
# مشاهدة أول 10 صفوف  
df.head(10)
```

```
# الاطلاع على الإحصاءات الوصفية  
df.describe()
```

```
# التحقق من أنواع البيانات والبحث عن قيم مفقودة  
df.info()
```

عند فحص `df.info()`، ستعرف عدد الصفوف الكلي، وما إذا كانت هناك أي أعمدة تحتوي على قيم ناقصة (NaN).

2. تنظيف وتجهيز البيانات (Data Cleaning & Preprocessing)

البيانات الخام نادراً ما تكون جاهزة للتحليل المباشر. في هذه المرحلة، نقوم بمعالجة المشاكل الشائعة مثل أخطاء الإدخال والتواريخ.

معالجة وتنسيق التواريخ

قد يتم استيراد التواريخ كأرقام تسلسلية (مثل نظام Excel). يمكننا تحويلها إلى صيغة تاريخ معتمدة عبر مكتبة Pandas:

```
df['Date'] = pd.to_datetime(df['Date'], unit='D', origin='1899-12-30')  
df['Date'].head()
```

تحديد وإصلاح القيم المفقودة (Missing Values) والشاذة (Outliers)

سنقوم بمعالجة الأخطاء في عمود التكلفة `Cost`. 1. تنظيف النصوص (إزالة الفراغات) وتحويلها إلى أرقام. 2. حساب المدى الربيعي (IQR) لتحديد القيم الشاذة. 3. استبدال القيم الناقصة والشاذة بالمتوسط الحسابي للقيم الطبيعية.

```
# تنظيف النصوص وتحويلها لقيم رقمية  
df['Cost'] = df['Cost'].astype(str).str.strip()  
df['Cost'] = pd.to_numeric(df['Cost'], errors='coerce')
```

```
# حساب الـ IQR لاكتشاف القيم الشاذة
```

```

Q1 = df['Cost'].quantile(0.25)
Q3 = df['Cost'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# حساب المتوسط باستثناء القيم الشاذة
mean_cost = df.loc[(df['Cost'] >= lower) & (df['Cost'] <= upper), 'Cost'].mean()

# استبدال القيم الناقصة والشاذة بالمتوسط
df['Cost'] = df['Cost'].apply(
    lambda x: mean_cost if pd.isna(x) or x < lower or x > upper else x
)

```

تصحيح الأخطاء الإملائية وتوحيد المعايير (Standardization)

نستخدم الدالة `replace` لإصلاح أخطاء الإدخال في عمودي المندوب والمتاجر.

```

# إصلاح خطأ في اسم المندوب
df['Rep'] = df['Rep'].replace('Mjeeed', 'Mjeed')

# توحيد أسماء المتاجر (استبدال Lulu Hyper بـ Lulu)
df['Store'] = df['Store'].replace('Lulu Hyper', 'Lulu')

# للتأكد من نجاح التعديل:
print(df['Rep'].value_counts())
print(df['Store'].value_counts())

```

3. هندسة الميزات (Feature Engineering)

هندسة الميزات تعني خلق متغيرات جديدة (أعمدة) من البيانات الموجودة لزيادة القدرة التحليلية.

إنشاء عمود الربح (Profit)

الربح هو ببساطة الفرق بين المبيعات والتكلفة.

```
df['Profit'] = df['Sale'] - df['Cost']
```

إضافة متغير الجنس (Gender)

استناداً إلى أسماء المناديب، سنضيف عموداً يحدد الجنس باستخدام قائمة بأسماء الإناث وتطبيق الدالة `apply` مع `lambda`.

```
Female = ['Maliha', 'Ghalia']
df['Gender'] = df['Rep'].apply(lambda x: "Female" if x in Female else "Male")
```

4. التحليل الإحصائي واختبار الفروض (Inferential Statistics)

هل هناك فرق حقيقي بين مبيعات الذكور والإناث، أم أنه مجرد صدفة؟ للإجابة، نستخدم اختبار `t` للعينات المستقلة (Independent T-Test).

```
from scipy import stats

# فصل الأرباح حسب الجنس
profit_male = df[df['Gender'] == 'Male']['Profit']
profit_female = df[df['Gender'] == 'Female']['Profit']

# إجراء الاختبار
t_stat, p_value = stats.ttest_ind(profit_male, profit_female)

print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")

# التفسير
if p_value < 0.05:
    print("النتيجة: يوجد فرق ذو دلالة إحصائية بين متوسط أرباح الذكور والإناث")
else:
    print("النتيجة: لا يوجد فرق ذو دلالة إحصائية، والفرق الظاهري يعود للصدفة العشوائية")
```

ملاحظة: في بياناتنا، ظهرت النتيجة $p\text{-value} = 0.739$ ، مما يعني أن أداء الجنسين متكافئ ولا يوجد فرق جوهري.

5. التجميع (Aggregation) والتمثيل البصري (Data Visualization)

يعد التمثيل البصري أفضل طريقة لعرض الرؤى للإدارة.

حساب المتوسط وعرضه في رسم شريطي بسيط

نستخدم `groupby` لحساب متوسط المبيعات لكل جنس، ثم نرسم النتيجة عبر `matplotlib`.

```
# حساب المتوسط
avg_Sales_Gender = df.groupby('Gender')['Sale'].mean()

# الرسم البياني
avg_Sales_Gender.plot(kind='bar', color=['pink', 'blue'])
plt.xlabel('Gender')
plt.ylabel('Average Sales')
plt.title('Average Sales by Gender')
plt.show()
```

دمج معيارين واستخدام الشروط

للوصول لمعلومات أكثر تفصيلاً، يمكن دمج الفلتر لحساب المبيعات بمدينة محددة وبناءً على شروط. يجب الانتباه لوضع الشروط بين أقواس `()` واستعمال `&` لـ (و) و `|` لـ (أو).

```
# مثال: المبيعات في الرياض التي تجاوز ربحها 1000
filtered_df = df[(df['City'] == 'Riyadh') & (df['Profit'] > 1000)]
```

الرسوم البيانية التفاعلية (Interactive Dashboards)

لعمل لوحات تحكم متقدمة، يمكننا استخدام مكتبة `Plotly` لإنشاء رسم يتضمن قائمة منسدلة تتيح التبديل بين المتاجر لعرض مبيعات المنتجات حسب المدينة (Dropdown).

```
import plotly.graph_objects as go

# 1. تجميع البيانات
grouped = df.groupby(['Store', 'Prod', 'City'])['Sale'].sum().reset_index()
stores = sorted(grouped['Store'].unique())
cities = sorted(grouped['City'].unique())
```

```

fig = go.Figure()

# 2. إنشاء المسارات (Traces)
for s_idx, store in enumerate(stores):
    df_s = grouped[grouped['Store'] == store]
    for c_idx, city in enumerate(cities):
        sub = df_s[df_s['City'] == city]
        fig.add_bar(
            x=sub['Prod'],
            y=sub['Sale'],
            name=city,
            visible=(s_idx == 0) # إظهار المتجر الأول فقط في البداية
        )

# 3. بناء أزرار القائمة المنسدلة
buttons = []
n_traces_per_store = len(cities)
total_traces = len(stores) * len(cities)

for s_idx, store in enumerate(stores):
    visible = [False] * total_traces
    start = s_idx * n_traces_per_store
    end = start + n_traces_per_store
    for i in range(start, end):
        visible[i] = True

    buttons.append(dict(
        label=store,
        method="update",
        args=[
            {"visible": visible},
            {"title": f"Product Sales by City - Store: {store}"}
        ]
    ))

# 4. إعدادات الشكل العام
fig.update_layout(
    title="Product Sales by City - Store: " + stores[0],
    xaxis_title="Product",
    yaxis_title="Total Sales",
    barmode="group",
    updatemenus=[dict(
        type="dropdown",
        x=1.15, y=1.1,

```

```
        showactive=True,  
        buttons=buttons  
    )],  
    legend=dict(title="City", font=dict(size=16))  
)  
  
# عرض الرسم البياني  
fig.show()  
  
# تصدير الرسم كملف HTML ليُعرض تفاعلياً في أي متصفح  
fig.write_html("product_sales_dashboard.html", include_plotlyjs="cdn")
```

يسمح لك تصدير الملف كـ **HTML** بمشاركته مع زملائك أو مديرك لعرض النتائج والتفاعل معها بسهولة.